

國立雲林科技大學
資訊管理系

線上購物系統

組長：B11123009 劉忠原

組員：A11323020 王涓靖

A11323029 陳品涵

A11323037 吳亭萱

A11323041 郭佳恩

目錄

壹、 需求蒐集.....	2
一、 功能性需求	2
二、 非功能性需求.....	2
三、 主要實體與屬性概述	3
貳、 概念設計.....	5
一、 ER 圖	5
二、 實體描述文件.....	5
參、 ER 模型到關聯網要的轉換	7
一、 關聯網要表	7
二、 關聯網要圖	10
肆、 正規化	11
一、 未正規化的資料(UNF)	11
二、 第一正規形式 (1NF)	11
三、 第二正規形式 (2NF)	12
四、 第三正規形式 (3NF)	12
伍、 SQL 撰寫.....	13
一、 DDL 腳本.....	13
二、 DML 腳本.....	14
三、 關鍵查詢的 SQL 查詢	15
陸、 資料載入.....	20
一、 資料載入腳本.....	20
二、 範例資料展示	20
三、 統計報告	21
柒、 測試與查詢優化	23
一、 功能性測試報告	23
二、 查詢效能報告(EXPLAIN 或 EXPLAIN ANALYZE)	24
三、 壓力測試結果.....	25

壹、需求蒐集

本專案旨在開發一個功能全面的線上購物系統，滿足顧客的購物需求，同時提升商家的管理效率。系統涵蓋從商品瀏覽、購物車結算到交易完成的完整流程，並提供供應商與員工的後台管理功能。系統關注用戶體驗，並著重於資料的整合與自動化處理，確保交易流程的順暢與庫存管理的精確性。

一、功能性需求

1. 顧客功能

- 帳戶管理：顧客可更新個人聯絡資訊（如電話與地址），以便準確處理訂單。
- 商品瀏覽與搜尋：提供分類瀏覽與搜尋功能，支援商品排序與篩選（例如價格或評分）。
- 購物車管理：顧客可以將商品加入購物車，並隨時更新商品數量或移除商品。
- 訂單提交與交易支付：支援完整的訂單提交與付款功能，顧客可查看歷史訂單資訊。
- 商品評論：顧客可撰寫商品評論並給予評分，幫助其他用戶參考。

2. 員工功能

- 訂單管理：員工可查看顧客訂單並處理訂單狀態（例如出貨或取消）。
- 資料分析：提供銷售數據與報表，供管理者參考營運績效。

3. 供應商功能

- 商品管理：供應商可以新增、更新或刪除商品資訊，包括價格、描述與庫存數量。
- 庫存管理：實時查看商品庫存，可幫助供應商進貨。

4. 系統功能

- 交易與支付管理：記錄每筆交易的詳細資訊，確保支付流程的安全性與準確性。
- 分類管理：支援商品多層級分類，方便顧客瀏覽與管理。

二、非功能性需求

1. 效能要求

- 查詢效能：系統應能夠在 2 秒內完成對 1,000 筆資料的查詢，無論是商品、訂單還是顧客資訊，複雜查詢或是聯結查詢應該不超過 5 秒。
- 寫入效能：在大量數據寫入的情況下，例如批量處理、定期數據匯入等，系統應能在 1 秒內處理至少 100 筆資料的寫入操作。
- 資料處理效能：在處理複雜的數據運算或統計分析時，系統應能在合理的時間內完成，例如計算過程不應超過 10 秒。

2. 資料一致性與完整性要求

- 資料完整性：資料庫應設置適當的約束，例如外鍵、唯一性約束等，保證資料關聯間的一致性，防止數據異常或錯誤插入。

- 資料驗證與清理：在資料輸入和更新過程中，系統應檢查資料的有效性與一致性，並進行必要的清理工作，避免不一致的資料進入資料庫。
3. 安全性要求
- 存取控制與權限管理：設置不同使用者角色，例如顧客、員工、供應商，並根據對資料的存取進行授權管理，確保只有授權用戶才能訪問敏感資料。
 - 加密技術：敏感資料如顧客資訊、支付資料等，應加密存儲與傳輸，以防止資料在儲存或傳輸過程中被竊取。

三、主要實體與屬性概述

1. 顧客(Customer)

屬性：顧客編號(customer_id)、身份證字號(identification_number)、姓名(name)、電話(phone)、電子郵件(email)、地址(address)、創建時間(account_creation_date)

描述：負責儲存所有用戶的基本資訊，例如個人聯絡方式。

2. 購物車(Shopping Cart)

屬性：購物車編號(shopping_cart_id)、商品數量(quantity)、創建時間(cart_creation_date)

描述：儲存顧客目前選購但尚未結帳的商品資訊。

3. 訂單(Order)

屬性：訂單編號(order_id)、訂單日期(date)、訂單狀態(status)、配送地址(address)、折扣(discount)

描述：用來管理顧客下的每一筆交易，包含訂單的日期、金額及相關配送資訊。

4. 訂單明細(Order Detail)

屬性：顧客編號(customer_id)、訂單編號(order_id)、商品數量(quantity)

描述：細分每個訂單中的商品資訊，記錄了單一訂單內購買的每件商品及其數量。

5. 商品(Product)

屬性：商品編號(product_id)、名稱(name)、價格(price)、商品描述(description)、上架時間(listing_date)

描述：記錄了線上購物系統中可以銷售的所有商品資訊，包含商品的名稱、價格、描述。

6. 供應商(Supplier)

屬性：供應商編號(supplier_id)、供應商名字(company_name)、地址(address)、聯絡人姓名(contact_name)、電話(phone)、電子郵件(email)

描述：管理所有提供商品的供應商資訊，包括聯絡方式及公司名稱。

7. 評論(Review)

屬性：顧客編號(customer_id)、商品編號(product_id)、商品評分(rating)、評論內容(content)、評論日期(date)

描述：記錄顧客對商品的評價，包括評論內容及評分。

8. 庫存(Inventory)

屬性：庫存編號(inventory_id)、商品庫存數量(quantity)、商品庫存位置(location)、更新日期(last_updated)

描述：管理商品的庫存數量及庫存位置，確保商品可以正常銷售。

9. 員工(Employee)

屬性：員工編號(employee_id)、姓名(name)、職位(position)、電話(phone)、電子郵件(email)、在職狀況(status)

描述：負責管理系統中所有工作人員的基本資訊及其職位相關信息。

10. 商品類別(Product Category)

屬性：商品類別編號(product_category_id)、類別名稱(name)

描述：用來記錄商品的類別及名稱。

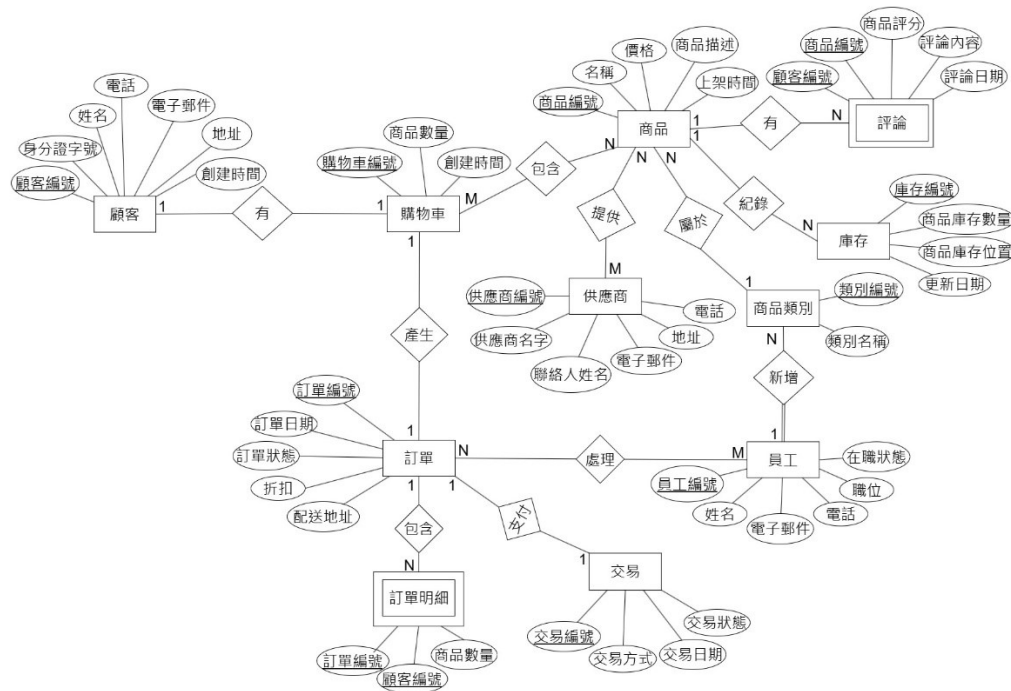
11. 交易(Transaction)

屬性：交易編號(transaction_id)、交易日期(date)、交易方式(method)、交易狀態(status)

描述：用來記錄每筆付款交易的詳細資訊，包括交易的日期、金額及支付方式。

貳、 概念設計

一、 ER 圖



二、 實體描述文件

1. 實體：Customer

主鍵：customer_id

實體之間的關係：與 Shopping Cart：一對多

2. 實體：Shopping Cart

主鍵：shopping_cart_id 外鍵：customer_id, product_id

實體之間的關係：與 Product：多對多；與 Order：一對一

3. 實體：Order

主鍵：order_id 外鍵：customer_id, employee_id

實體之間的關係：與 Employee：多對多

4. 實體：Order Detail

主鍵：customer_id+order_id 外鍵：customer_id, order_id, product_id

實體之間的關係：與 Order：一對多

5. 實體：Product

主鍵：product_id 外鍵：product_category_id, supplier_id

實體之間的關係：與 Review：一對多

6. 實體：Supplier
主鍵：supplier_id
實體之間的關係：與 Product：多對多
7. 實體：Review
主鍵：customer_id+product_id 外鍵：customer_id, product_id
實體之間的關係：與 Customer：多對一
8. 實體：Inventory
主鍵：inventory_id 外鍵：product_id
實體之間的關係：與 Product：多對多
9. 實體：Employee
主鍵：employee_id
實體之間的關係：與 Product Category：一對多
10. 實體：Product Category
主鍵：product_category_id 外鍵：product_id
實體之間的關係：與 Product：一對多
11. 實體：Transaction
主鍵：transaction_id 外鍵：customer_id, order_id
實體之間的關係：與 Order：一對一

參、 ER 模型到關聯網要的轉換

一、關聯網要表

- PRIMARY KEY：用來唯一標識表中的每一筆資料，確保該欄位的值不會重複，且不能為空。
- FOREIGN KEY：來維護資料表之間的參考完整性，防止子表中的外鍵值指向主表中不存在的主鍵值，透過外鍵的約束，確保兩個資料表的關聯是一致的。
ON UPDATE NO ACTION /ON DELETE NO ACTION 當主表的主鍵被嘗試更新或刪除時，如果有子表的外鍵依賴於該主鍵，則不允許更新或刪除(會拋出錯誤)。
- UNIQUE：確保欄位中的值必須是唯一的，不能有重複值，例如，phone、email。
- NOT NULL：欄位中的每一筆資料都必須有值，不能為空值，用來維護資料的完整性。
- CHECK：用來確保資料符合特定條件，例如商品價格要大於 0 元。

Customer		
Column Name	Data Type	Constraint
customer_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
identification_number	TEXT	UNIQUE NOT NULL
name	TEXT	NOT NULL
phone	TEXT	UNIQUE NOT NULL
email	TEXT	UNIQUE NOT NULL
address	TEXT	NOT NULL
account_creation_date	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

Employee		
Column Name	Data Type	Constraint
employee_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
name	TEXT	NOT NULL
phone	TEXT	UNIQUE NOT NULL
email	TEXT	UNIQUE NOT NULL
position	TEXT	NOT NULL
status	TEXT	NOT NULL

Product Category		
Column Name	Data Type	Constraint
product_category_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
name	TEXT	NOT NULL

Supplier		
Column Name	Data Type	Constraint
supplier_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
company_name	TEXT	NOT NULL
contact_name	TEXT	NOT NULL
phone	TEXT	UNIQUE NOT NULL
email	TEXT	UNIQUE NOT NULL
address	TEXT	NOT NULL

Order		
Column Name	Data Type	Constraint
order_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
customer_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
employee_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
shopping_cart_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
date	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP
status	TEXT	NOT NULL
address	TEXT	NOT NULL
discount	INTEGER	NOT NULL

Transaction		
Column Name	Data Type	Constraint
transaction_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
customer_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
order_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
date	TIMESTAMP	NOT NULL
method	TEXT	NOT NULL
status	TEXT	NOT NULL

Shopping Cart		
Column Name	Data Type	Constraint
shopping_cart_id	INTEGER	PRIMARY KEY NOT NULL
customer_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
product_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
quantity	INTEGER	NOT NULL CHECK(quantity >0)
cart_creation_date	TIMESTAMP	NOT NULL

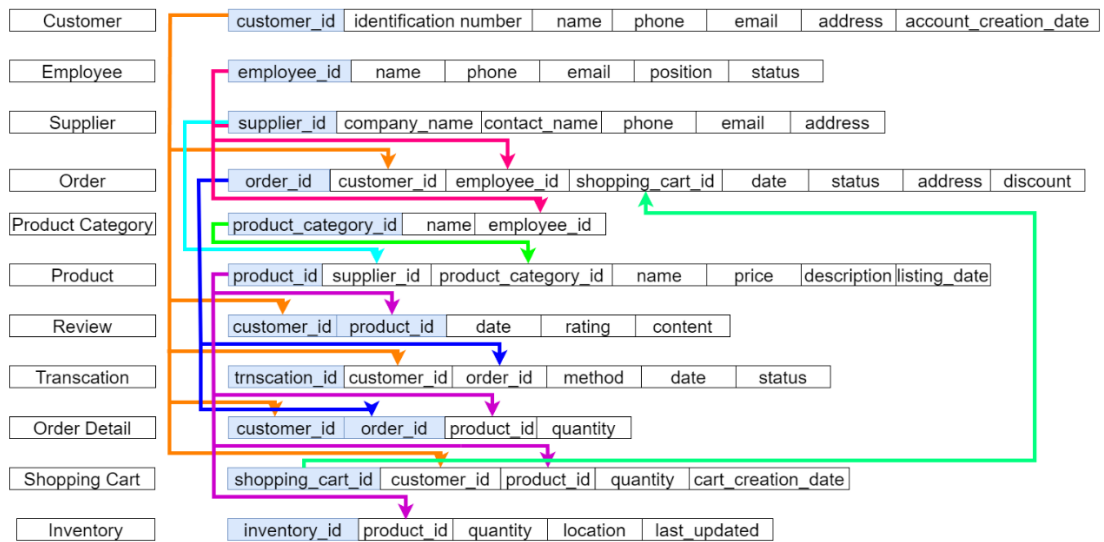
Order Detail		
Column Name	Data Type	Constraint
customer_id	INTEGER	PRIMARY KEY FOREIGN KEY NOT NULL
order_id	INTEGER	PRIMARY KEY FOREIGN KEY NOT NULL
product_id	INTEGER	FOREIGN NOT NULL
quantity	INTEGER	NOT NULL

Product		
Column Name	Data Type	Constraint
product_id	INTEGER	PRIMARY KEY SERIAL NOT NULL
supplier_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
product_category_id	INTEGER	FOREIGN KEY SERIAL NOT NULL
name	TEXT	NOT NULL
price	INTEGER	NOT NULL CHECK(price>0)
description	TEXT	NOT NULL
listing_date	TIMESTAMP	NOT NULL

Review		
Column Name	Data Type	Constraint
customer_id	INTEGER	PRIMARY KEY FOREIGN KEY NOT NULL
product_id	INTEGER	PRIMARY KEY FOREIGN KEY NOT NULL
rating	INTEGER	NOT NULL CHECK(rating >=1 AND rating <=5)
content	TEXT	NOT NULL
date	TIMESTAMP	NOT NULL

Inventory		
Column Name	Data Type	Constraint
inventory_id	INTEGER	PRIMARY KEY NOT NULL
product_id	INTEGER	FOREIGN KEY NOT NULL
quantity	INTEGER	NOT NULL
location	TEXT	NOT NULL
last_updated	TIMESTAMP	NOT NULL

二、關聯網要圖



肆、 正規化

一、未正規化的資料(UNF)

以下為未正規化的資料，未正規化會造成資料重複、依賴混亂和多值欄位，導致資料結構複雜、查詢困難，並且容易產生一致性問題。

顧客編號	姓名	商品名稱	數量	價格	供應商名字	評分	評論內容	訂單編號	員工姓名	訂單狀態	庫存位置	庫存數量	交易方式	購物車編號
1	王小明	衛生紙	2	350	清新實業	4	超讚	101	林發財	已完成	A 倉庫	10	信用卡	1
1	王小明	手套	1	180	金盛公司	5	超讚	101	邱仲威	已完成	B 倉庫	30	貨到付款	2
2	陳淑芬	保溫杯	3	699	永豐公司	3	普通	102	張志岳	已完成	C 倉庫	8	行動支付	3

二、第一正規形式 (1NF)

目標：

- 確保所有欄位的值是原子值。
- 消除重複欄位，保證每個欄位只存放單一值。
- 把不同實體的資料分離成獨立的表。

拆解後的資料表：

1. 顧客表(顧客編號，姓名，電話，電子郵件，配送地址)
2. 購物車表(購物車編號，顧客編號，商品編號，數量)
3. 訂單表(訂單編號，顧客編號，員工編號，購物車編號，訂單日期，狀態，配送位置)
4. 訂單明細表(顧客編號，商品編號，商品編號，數量)
5. 商品表(商品編號，顧客編號，名稱，價格，商品類別，供應商編號，供應商名字，商品描述)
6. 評論表(顧客編號，商品編號，評論內容，商品評分)
7. 員工表(員工編號，姓名，電話，電子郵件，職位，在職狀況)
8. 庫存表(庫存編號，庫存位置，庫存數量)
9. 交易表(交易編號，顧客編號，交易日期，交易方式，交易狀態)

三、第二正規形式 (2NF)

目標：消除部分函數相依，確保每個非主鍵欄位完全依賴於整個主鍵，而不是僅依賴於主鍵的一部分。

問題 1：商品表中的商品類別欄位依賴於商品編號，存在函數相依，應該將商品類別與商品分開儲存。

商品類別的拆解：商品表中的商品類別欄位拆解為商品類別表，商品類別編號由商品表透過外鍵參照。

拆解後的資料表：

1. 商品表(商品編號，商品名稱，價格，商品描述，供應商名字，商品類別編號)
2. 商品類別表(商品類別編號，類別名稱)

四、第三正規形式 (3NF)

目標：消除傳遞依賴，確保所有非主鍵欄位直接依賴於主鍵。

問題：在商品表中供應商名字依賴於供應商編號，但供應商編號並不依賴於商品編號，存在傳遞依賴。

供應商的拆解：創建一個供應商表，以供應商編號作為主鍵，在商品表中去除供應商名字欄位，並以供應商編號作為外鍵，建立兩個表的關聯。

拆解後的資料表：

1. 商品表(商品編號，供應商編號，商品類別編號，商品名稱，價格，商品描述)
2. 供應商表(供應商編號，供應商名字，聯絡人姓名，電話，電子郵件，地址)

最後拆解完的資料表：

1. 顧客表(顧客編號，姓名，電話，電子郵件，配送地址)
2. 購物車表(購物車編號，顧客編號，商品編號，數量)
3. 訂單表(訂單編號，顧客編號，購物車編號，員工編號，訂單日期，訂單狀態，配送位置，折扣)
4. 訂單明細表(顧客編號，商品編號，數量)
5. 商品表(商品編號，供應商編號，商品類別編號，商品名稱，價格，商品描述)
6. 評論表(顧客編號，商品編號，評論內容，商品評分)
7. 員工表(員工編號，姓名，電話，電子郵件，職位，在職狀況)
8. 庫存表(庫存編號，庫存位置，庫存數量)
9. 商品類別表(商品類別編號，類別名稱)
10. 交易表(交易編號，交易日期，交易方式，交易狀態)
11. 供應商表(供應商編號，供應商名字，聯絡人姓名，電話，電子郵件，地址)

伍、SQL 撰寫

一、DDL 腳本

1. customer 資料表

```
data=# \d customer
          欄位          |          型別          | 資料表 "public.customer" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
customer_id | integer | | not null | nextval('customer_customer_id_seq'::regclass)
identification_number | text | | not null | 
name | text | | not null | 
phone | text | | not null | 
email | text | | not null | 
address | text | | not null | 
account_creation_date | timestamp without time zone | | not null | CURRENT_TIMESTAMP
索引:
    "customer_pkey" PRIMARY KEY, btree (customer_id)
    "customer_email_key" UNIQUE CONSTRAINT, btree (email)
    "customer_phone_key" UNIQUE CONSTRAINT, btree (phone)
    "identification_number" UNIQUE CONSTRAINT, btree (identification_number)
檢查規則:
    "email" CHECK (email ~ '^[a-zA-Z0-9_+]{1,64}[a-zA-Z0-9-]{1,4}@[a-zA-Z]{2,5}.$')::text)
參考表:
    TABLE "order" CONSTRAINT "customer" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    TABLE "order_detail" CONSTRAINT "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    TABLE "shopping_cart" CONSTRAINT "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    TABLE "review" CONSTRAINT "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    TABLE "transaction" CONSTRAINT "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
data=# INSERT INTO "public"."customer" ("identification_number", "name", "phone", "email", "address") VALUES ('L123456789', '王周斌', '0987654321', 'not_email', '高雄市鳥松區平路66號');
ERROR:  new row for relation "customer" violates check constraint "email"
詳細資料:  Failing row contains (1003, L123456789, 王周斌, 0987654321, not_email, 高雄市鳥松區平路66號, 2024-12-17 03:02:32.711303).
```

2. employee 資料表

```
data=# INSERT INTO "public"."customer" ("identification_number", "name", "phone", "email", "address") VALUES ('L123456789', '王周斌', '0987654321', 'not_email', '高雄市鳥松區平路66號');
ERROR:  new row for relation "customer" violates check constraint "email"
詳細資料:  Failing row contains (1003, L123456789, 王周斌, 0987654321, not_email, 高雄市鳥松區平路66號, 2024-12-17 03:02:32.711303).
data=# \d employee
          欄位          |          型別          | 資料表 "public.employee" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
employee_id | integer | | not null | nextval('employee_employee_id_seq'::regclass)
name | text | | not null | 
phone | text | | not null | 
email | text | | not null | 
position | text | | not null | 
status | integer | | not null | 
索引:
    "employee_pkey" PRIMARY KEY, btree (employee_id)
    "employee_email_key" UNIQUE CONSTRAINT, btree (email)
    "employee_phone_key" UNIQUE CONSTRAINT, btree (phone)
檢查規則:
    "email" CHECK (email ~ '^[a-zA-Z0-9_+]{1,64}[a-zA-Z0-9-]{1,4}@[a-zA-Z]{2,5}.$')::text)
參考表:
    TABLE "order" CONSTRAINT "employee" FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
data=# INSERT INTO "public"."employee" ("name", "phone", "email", "position", "status") VALUES ('王周斌', '0987654321', 'not_email', '實習生', 1);
ERROR:  new row for relation "employee" violates check constraint "email"
詳細資料:  Failing row contains (102, 王周斌, 0987654321, not_email, 實習生, 1).
```

3. inventory 資料表

```
data=# \d inventory
          欄位          |          型別          | 資料表 "public.inventory" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
inventory_id | integer | | not null | nextval('inventory_inventory_id_seq'::regclass)
product_id | integer | | not null | 
quantity | integer | | not null | 
location | text | | not null | 
last_updated | timestamp without time zone | | not null | 
索引:
    "inventorys_pkey" PRIMARY KEY, btree (inventory_id)
外鍵檢查規則:
    "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
```

4. order 資料表

```
data=# \d order
          欄位          |          型別          | 資料表 "public.order" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
order_id | integer | | not null | nextval('order_order_id_seq'::regclass)
customer_id | integer | | not null | nextval('order_customer_id_seq'::regclass)
employee_id | integer | | not null | nextval('order_employee_id_seq'::regclass)
shopping_cart_id | integer | | not null | nextval('order_shopping_cart_id_seq'::regclass)
date | timestamp without time zone | | not null | CURRENT_TIMESTAMP
status | text | | not null | 
discount | integer | | not null | 
索引:
    "order_pkey" PRIMARY KEY, btree (order_id)
外鍵檢查規則:
    "customer" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    "employee" FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
    "shopping_cart" FOREIGN KEY (shopping_cart_id) REFERENCES shopping_cart(shopping_cart_id)
```

5. order detail 資料表

```
data=# \d order_detail
          欄位          |          型別          | 資料表 "public.order_detail" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
customer_id | integer | | not null | 
order_id | integer | | not null | 
product_id | integer | | not null | 
quantity | integer | | not null | 
索引:
    "order_detail_pkey" PRIMARY KEY, btree (customer_id)
外鍵檢查規則:
    "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    "order_id" FOREIGN KEY (order_id) REFERENCES "order" (order_id)
    "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
```

6. product 資料表

```
data=# \d product
          欄位          |          型別          | 資料表 "public.product" | 是否 | 能否為 NULL | 預設
-----|-----|-----|-----|-----|-----
product_id | integer | | not null | nextval('product_product_id_seq'::regclass)
supplier_id | integer | | not null | 
product_category_id | integer | | not null | 
name | text | | not null | 
price | integer | | not null | 
description | text | | not null | 
listing_date | timestamp without time zone | | not null | 
索引:
    "product_pkey" PRIMARY KEY, btree (product_id)
檢查規則:
    "price" CHECK (price > 0)
外鍵檢查規則:
    "product_category" FOREIGN KEY (product_category_id) REFERENCES product_category(product_category_id)
    "supplier" FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)
參考表:
    TABLE "order_detail" CONSTRAINT "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
    TABLE "shopping_cart" CONSTRAINT "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
    TABLE "inventory" CONSTRAINT "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
    TABLE "review" CONSTRAINT "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
data=# INSERT INTO "public"."product" ("supplier_id", "product_category_id", "name", "price", "description", "listing_date") VALUES (5, 97, '測試商品', -1, '測試', '2024-12-17 03:09:56');
ERROR:  new row for relation "product" violates check constraint "price"
詳細資料:  Failing row contains (104, 5, 97, 測試商品, -1, 測試, 2024-12-17 03:09:56).
```

7. product_category 資料表

```
data=# \d product_category
          資料表 "public.product_category"
+-----+
| 欄位 | 型別 | 定序 | 能否為 NULL | 預設 |
+-----+
| product_category_id | integer | | not null | nextval('product_category_product_category_id_seq'::regclass) |
| name | text | | not null | |
+-----+
索引:
"product_category_pkey" PRIMARY KEY, btree (product_category_id)
```

8. review 資料表

```
data=# \d review
          資料表 "public.review"
+-----+
| 欄位 | 型別 | 定序 | 能否為 NULL | 預設 |
+-----+
| customer_id | integer | | not null | |
| product_id | integer | | not null | |
| date | timestamp without time zone | | not null | |
| rating | integer | | not null | |
| content | text | | not null | |
+-----+
索引:
"review_pkey" PRIMARY KEY, btree (customer_id, product_id)
檢查規則:
"rating" CHECK (rating >= 1 AND rating <= 5)
外鍵檢查規則:
"customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
"product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)

data=# INSERT INTO "public"."review" ("customer_id","product_id","date","rating","content") VALUES ('S','S','2024-12-17 03:09:56',-1,'超極差评');
ERROR: new row for relation "review" violates check constraint "rating"
詳細資料: Failing row contains (S, S, 2024-12-17 03:09:56, -1, 超極差评).
```

9. shopping cart 資料表

```
data=# \d shopping_cart
          資料表 "public.shopping_cart"
+-----+
| 欄位 | 型別 | 定序 | 能否為 NULL | 預設 |
+-----+
| shopping_cart_id | integer | | not null | nextval('shopping_cart_shopping_cart_id_seq'::regclass) |
| customer_id | integer | | not null | |
| product_id | integer | | not null | |
| quantity | integer | | not null | |
| cart_creation_date | timestamp without time zone | | not null | CURRENT_TIMESTAMP |
+-----+
索引:
"shopping_cart_pkey" PRIMARY KEY, btree (shopping_cart_id)
"shopping_cart_customer_id_key" UNIQUE CONSTRAINT, btree (customer_id)
檢查規則:
"quantity" CHECK (quantity > 0)
外鍵檢查規則:
"customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
"product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)
參考表:
TABLE "order" CONSTRAINT "shopping_cart" FOREIGN KEY (shopping_cart_id) REFERENCES shopping_cart(shopping_cart_id)

data=# INSERT INTO "public"."shopping_cart" ("customer_id","product_id","quantity") VALUES ('S','S','-1');
ERROR: new row for relation "shopping_cart" violates check constraint "quantity"
詳細資料: Failing row contains (103, S, S, -1, 2024-12-17 03:14:22.457227).
```

10. supplier 資料表

```
data=# \d supplier
          資料表 "public.supplier"
+-----+
| 欄位 | 型別 | 定序 | 能否為 NULL | 預設 |
+-----+
| supplier_id | integer | | not null | nextval('supplier_supplier_id_seq'::regclass) |
| company_name | text | | not null | |
| contact_name | text | | not null | |
| phone | text | | not null | |
| email | text | | not null | |
| address | text | | not null | |
+-----+
索引:
"supplier_pkey" PRIMARY KEY, btree (supplier_id)
"supplier_email_key" UNIQUE CONSTRAINT, btree (email)
"supplier_phone_key" UNIQUE CONSTRAINT, btree (phone)
```

11. transaction 資料表

```
data=# \d transaction
          資料表 "public.transaction"
+-----+
| 欄位 | 型別 | 定序 | 能否為 NULL | 預設 |
+-----+
| transaction_id | integer | | not null | nextval('transaction_transaction_id_seq'::regclass) |
| customer_id | integer | | not null | nextval('transaction_customer_id_seq'::regclass) |
| order_id | integer | | not null | nextval('transaction_order_id_seq'::regclass) |
| method | text | | not null | |
| date | timestamp without time zone | | not null | |
| status | text | | not null | |
+-----+
索引:
"transaction_pkey" PRIMARY KEY, btree (transaction_id)
外鍵檢查規則:
"customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
"order_id" FOREIGN KEY (order_id) REFERENCES "order"(order_id)
```

二、DML 腳本

1. customer 資料表

```
data=# INSERT INTO "customer" ("identification_number", "name", "phone", "email", "address") VALUES ('A123456789', '測試用戶', '0912345678', 'test@example.com', '台北市測試路123號');
INSERT 0 1
data=# SELECT * FROM "customer" WHERE "identification_number"='A123456789';
 customer_id | identification_number | name | phone | email | address | account_creation_date
+-----+
| 107 | A123456789 | 測試用戶 | 0912345678 | test@example.com | 台北市測試路123號 | 2024-12-16 18:11:06.921481
(1 筆資料)

data=# INSERT INTO "customer" ("identification_number", "name", "phone", "email", "address") VALUES ('A123456789', '測試用戶', '0912345678', 'test@example.com', '台北市測試路123號');
ERROR: duplicate key value violates unique constraint "identification_number"
詳細資料: Key (identification_number)=(A123456789) already exists.
data=# UPDATE "customer" SET "phone" = '0987654321', "email" = 'updated@example.com' WHERE "identification_number" = 'A123456789';
UPDATE 1
data=# SELECT * FROM "customer" WHERE "identification_number"='A123456789';
 customer_id | identification_number | name | phone | email | address | account_creation_date
+-----+
| 107 | A123456789 | 測試用戶 | 0987654321 | updated@example.com | 台北市測試路123號 | 2024-12-16 18:11:06.921481
(1 筆資料)

data=# DELETE FROM "customer" WHERE "identification_number" = 'A123456789';
DELETE 1
data=# SELECT * FROM "customer" WHERE "identification_number"='A123456789';
 customer_id | identification_number | name | phone | email | address | account_creation_date
+-----+
(0 筆資料)
```

2. inventory 資料表

```
datal=# INSERT INTO "inventory" ("product_id", "quantity", "location") VALUES (115, 100, '倉庫A');
INSERT 0 1
datal=# SELECT * FROM "inventory" WHERE "location"='倉庫A' AND "product_id"=115;
 inventory_id | product_id | quantity | location | last_updated
-----
139 | 115 | 100 | 倉庫A | 2024-12-16 18:22:23.46913
(1 筆資料)
```

```
datal=# INSERT INTO "inventory" ("product_id", "quantity", "location") VALUES (9999, 50, '倉庫B');
ERROR: insert or update on table "inventory" violates foreign key constraint "product_id"
詳細資料: Key (product_id)=(9999) is not present in table "product".
datal=#
datal=# UPDATE "inventory" SET "quantity" = 150 WHERE "location"='倉庫A' AND "product_id"=115;
UPDATE 1
datal=# SELECT * FROM "inventory" WHERE "location"='倉庫A' AND "product_id"=115;
 inventory_id | product_id | quantity | location | last_updated
-----
139 | 115 | 150 | 倉庫A | 2024-12-16 18:22:23.46913
(1 筆資料)
```

```
datal=# DELETE FROM "inventory" WHERE "location"='倉庫A' AND "product_id"=115;
DELETE 1
datal=# SELECT * FROM "inventory" WHERE "location"='倉庫A' AND "product_id"=115;
 inventory_id | product_id | quantity | location | last_updated
-----
(0 筆資料)
```

三、關鍵查詢的 SQL 查詢

1. 查詢人數大於 50 且地址在台北、台中、台南以及台東的資料

解釋：計算地址在台北、台中、台南以及台東的人數(大於 50)，且查詢的結果會根據地區進行分類與排列。

```
datal=# -- 1. 找出人數大於50且地址在臺北、臺中、臺南以及臺東的資料；
datal=# SELECT SUBSTRING(address FROM 1 FOR 3) AS region, COUNT(*) AS customer_count
datal=# FROM "customer"
datal=# WHERE address LIKE '臺北%' OR address LIKE '臺中%' OR address LIKE '臺南%' OR address LIKE '臺東%'
datal=# GROUP BY region
datal=# HAVING COUNT(*) > 50
datal=# ORDER BY region;
 region | customer_count
-----
臺中市 | 140
臺北市 | 54
臺南市 | 85
(3 筆資料)
```

2. 查詢信用卡交易成功次數最多的前五名顧客

解釋：統計使用信用卡進行交易且交易狀態為「成功」的次數，查詢結果會按交易數降序排列，並顯示顧客編號及成功交易次數。

```
datal=# -- 找出前五名使用信用卡交易成功次數最多的顧客。
datal=# SELECT public.customer.customer_id, COUNT(transaction.transaction_id) AS total_success_transactions
datal=# FROM public.customer
datal=# JOIN public.transaction ON public.customer.customer_id = public.transaction.customer_id
datal=# WHERE public.transaction.method = '信用卡' AND public.transaction.status = '成功'
datal=# GROUP BY public.customer.customer_id
datal=# ORDER BY total_success_transactions DESC
datal=# LIMIT 5;
 customer_id | total_success_transactions
-----
800 | 4
126 | 1
195 | 1
220 | 1
119 | 1
(5 筆資料)
```

3. 取得在職員工處理的訂單數量排名

解釋：透過 JOIN 將訂單表與員工表連接，篩選出狀態為 1（在職）的員工，統計每位員工處理的訂單數量，並依據總數由大到小排序。

```
datal=# -- 取得在職員工處理的訂單數量排名。
datal=# SELECT public.order.employee_id, COUNT(public.order.order_id) AS order_count
datal=# FROM public.order
datal=# JOIN public.employee ON public.order.employee_id = public.employee.employee_id
datal=# WHERE public.employee.status = 1
datal=# GROUP BY public.order.employee_id
datal=# ORDER BY order_count DESC
datal=# LIMIT 20;
 employee_id | order_count
-----
95 | 6
60 | 5
25 | 5
81 | 5
4 | 5
23 | 4
49 | 4
35 | 4
43 | 4
99 | 4
11 | 3
18 | 3
21 | 3
77 | 3
15 | 3
58 | 3
29 | 3
32 | 3
41 | 3
5 | 3
(20 筆資料)
```

4. 查詢上架至少 10 項商品的供應商

解釋：統計每個供應商上架商品的數量，篩選數量大於等於 10 的供應商，搜尋結果按照商品數量降序排列。

```
data=# -- 4. 找出在平台上架至少10項商品的供應商且要根據數量多寡排名：
data=# SELECT s.supplier_id, COUNT(p.product_id) AS product_count
data=# FROM "product" p
data=# JOIN "supplier" s ON p.supplier_id = s.supplier_id
data=# GROUP BY s.supplier_id
data=# HAVING COUNT(p.product_id) >= 10
data=# ORDER BY product_count DESC;
supplier_id | product_count
-----
93          | 12
118         | 11
72          | 10
104         | 10
94          | 10
(5 筆資料)
```

5. 查詢評分至少 4 分且被評價至少 3 次的商品

解釋：使用子查詢找出評分大於等於 4 且被評價次數大於等於 3 的商品，搜尋結果按商品編號排序。

```
data=# -- 5. 找出評分至少4分且被評價過至少3次的商品：
data=# SELECT r.product_id
data=# FROM "review" r
data=# GROUP BY r.product_id
data=# HAVING AVG(r.rating) >= 4 AND COUNT(r.product_id) >= 3
data=# ORDER BY r.product_id
data=# LIMIT 20;
product_id
-----
13
48
56
65
73
76
95
102
104
107
109
140
154
164
168
180
190
209
220
223
(20 筆資料)
```

6. 查詢總銷售額前三名的商品類別

解釋：透過 JOIN 連接商品類別表、商品表和訂單明細表，計算每個商品類別的總銷售額，篩選出銷售額最高的前三名類別。

```
data=# -- 6. 找出總銷售額前三名的商品類別：
data=# SELECT pc.name, SUM(od.quantity * p.price) AS total_sales
data=# FROM "order_detail" od
data=# JOIN "product" p ON od.product_id = p.product_id
data=# JOIN "product_category" pc ON p.product_category_id = pc.product_category_id
data=# GROUP BY pc.name
data=# ORDER BY total_sales DESC
data=# LIMIT 3;
name | total_sales
-----
工具和家庭改善 | 92548
藝術、工藝和縫紉 | 81938
食品雜貨 | 60331
(3 筆資料)
```

7. 查詢未撰寫評價內容的顧客

解釋：利用 JOIN 和 LEFT JOIN 連接顧客表、訂單表及評價表，篩選出下過訂單但未撰寫任何評價的顧客資料。

```
data=# -- 7. 取得所有沒有撰寫評價內容的顧客資料：
data=# SELECT c.customer_id, c.name
data=# FROM "customer" c
data=# LEFT JOIN "review" r ON c.customer_id = r.customer_id
data=# WHERE r.content IS NULL
data=# LIMIT 10;
customer_id | name
-----
251 | 周淳淵
681 | 湯昭晴
151 | 李元皓
253 | 歐佑琪
518 | 蔣衛國
339 | 黃勝國
27 | 袁寶常
882 | 柯勛知
834 | 鄧芝誠
178 | 黃常彥
(10 筆資料)
```

8. 查詢庫存量最低的 5 個商品與其供應商

解釋：JOIN 商品表、庫存表與供應商表，依庫存量升序排列，並使用 LIMIT 5 篩選庫存量最低的商品及其供應商。

```
data=# -- 8. 查詢庫存量最低的5個商品與其供應商：
data=# SELECT p.name AS product_name, s.company_name AS supplier_name, i.quantity
data=# FROM "inventory" i
data=# JOIN "product" p ON i.product_id = p.product_id
data=# JOIN "supplier" s ON p.supplier_id = s.supplier_id
data=# ORDER BY i.quantity ASC
data=# LIMIT 5;
 product_name | supplier_name | quantity
-----
空氣清淨機   | 博大機械科技有限公司 | 1
吸塵器       | 長青農業科技股份有限公司 | 1
空氣清淨機   | 宏達製造股份有限公司 | 2
電風扇       | 宏瑞國際貿易有限公司 | 2
電風扇       | 瑞益金融控股有限公司 | 3
(5 筆資料)
```

9. 查詢顧客編號為 73 的購物車商品與數量

解釋：透過子查詢在購物車表中找出顧客編號 73 的購物車資料，並使用聚合函數計算每個商品的總數量，最後顯示商品編號及各個商品的總數量。

```
data=# -- 9. 查詢顧客編號73的購物車商品與其數量：
data=# SELECT sc.product_id, SUM(sc.quantity) AS total_quantity
data=# FROM "shopping_cart" sc
data=# WHERE sc.customer_id = 73
data=# GROUP BY sc.product_id;
 product_id | total_quantity
-----
          532 | 2
(1 筆資料)
```

10. 查詢近七天總銷售額前三的商品

解釋：JOIN 訂單表、訂單明細表及商品表，篩選近七天的訂單，計算每個商品的銷售總額，結果按總銷售額降序排列，並用 LIMIT 3 限制結果。

```
data=# -- 10. 查詢近七天總銷售額前三的商品：
data=# SELECT p.name, SUM(od.quantity * p.price) AS total_sales
data=# FROM "order_detail" od
data=# JOIN "product" p ON od.product_id = p.product_id
data=# JOIN "order" o ON od.order_id = o.order_id
data=# WHERE o.date >= CURRENT_DATE - INTERVAL '60 days'
data=# GROUP BY p.name
data=# ORDER BY total_sales DESC
data=# LIMIT 3;
 name | total_sales
-----
電視  | 14168
按摩椅 | 6498
電動牙刷 | 5816
(3 筆資料)
```

11. 查詢每個供應商供應的商品總數量及平均價格

解釋：透過 JOIN 連接商品表、供應商表，計算每個供應商的商品數量與商品的平均價格，並用 GROUP BY 按供應商編號分組，最後返回最多 10 個供應商的結果。

```
data=# -- 11. 查詢每個供應商供應的商品總數量及平均價格：
data=# SELECT s.company_name,
data=# COUNT(p.product_id) AS total_products,
data=# AVG(p.price) AS average_price
data=# FROM "product" p
data=# JOIN "supplier" s ON p.supplier_id = s.supplier_id
data=# GROUP BY s.supplier_id
data=# LIMIT 10;
 company_name | total_products | average_price
-----
宏達製造股份有限公司 | 4 | 2119.2500000000000000
白特廣告設計有限公司 | 3 | 1543.3333333333333333
博瑞生技股份有限公司 | 6 | 1781.3333333333333333
鼎裕食品有限公司 | 2 | 211.5000000000000000
欣華教育發展有限公司 | 6 | 1247.0000000000000000
安邦安全科技有限公司 | 2 | 2115.5000000000000000
博大機械科技有限公司 | 5 | 1141.0000000000000000
華泰機械科技有限公司 | 6 | 1186.3333333333333333
巨峰精密科技有限公司 | 6 | 1124.3333333333333333
華光光電股份有限公司 | 3 | 666.0000000000000000
(10 筆資料)
```

12. 查詢所有顧客的訂單總數量和總金額

解釋：透過 JOIN 連接訂單表、訂單明細表及商品表，計算每位顧客的訂單數量與消費總金額，並用 GROUP BY 按顧客編號分組。

```
datal=# -- 12. 查詢所有顧客的訂單總數量和總金額：
datal=# SELECT o.customer_id, COUNT(o.order_id) AS order_count, SUM(od.quantity * p.price) AS total_amount
datal=# FROM "order" o
datal=# JOIN "order_detail" od ON o.order_id = od.order_id
datal=# JOIN "product" p ON od.product_id = p.product_id
datal=# GROUP BY o.customer_id
datal=# LIMIT 20;
customer_id | order_count | total_amount
-----
788 | 1 | 14886
684 | 2 | 5814
229 | 1 | 2925
68 | 2 | 12798
481 | 1 | 1296
607 | 1 | 7216
258 | 1 | 6928
162 | 1 | 3076
341 | 1 | 10674
738 | 1 | 854
364 | 1 | 4270
762 | 1 | 4280
169 | 1 | 1102
799 | 1 | 6930
180 | 1 | 1004
323 | 1 | 5523
379 | 1 | 6048
509 | 1 | 780
114 | 1 | 1544
415 | 1 | 364
(20 筆資料)
```

13. 查詢過去一週內未購買商品的顧客

解釋：透過子查詢找出過去一週內有購買紀錄的顧客，再用 NOT IN 篩選未出現在子查詢結果中的顧客資料。

```
datal=# -- 13. 查詢過去一週內未購買商品的顧客：
datal=# SELECT c.customer_id, c.name
datal=# FROM "customer" c
datal=# WHERE c.customer_id NOT IN (
datal=#     SELECT DISTINCT o.customer_id
datal=#     FROM "order" o
datal=#     JOIN "order_detail" od ON o.order_id = od.order_id
datal=#     JOIN "product" p ON od.product_id = p.product_id
datal=#     WHERE o.date >= CURRENT_DATE - INTERVAL '7 days'
datal=# )LIMIT 10;
customer_id | name
-----
2 | 周育盈
3 | 楊慧永
4 | 黃梅歌
5 | 張志月
6 | 阮煒葉
7 | 楊梓江
8 | 鄒卓馳
9 | 卓希聲
10 | 陳君鑫
11 | 錢環眾
(10 筆資料)
```

14. 查詢購物車總金額超過 1 萬的顧客編號與購物車編號

解釋：透過子查詢計算每個購物車的總金額（商品價格 × 數量），篩選金額超過 1 萬的購物車，顯示顧客編號及購物車編號。

```
datal=# -- 15. 查詢購物車總金額超過1萬的顧客編號與購物車編號：
datal=# SELECT sc.customer_id, sc.shopping_cart_id, SUM(sc.quantity * p.price) AS total_amount
datal=# FROM "shopping_cart" sc
datal=# JOIN "product" p ON sc.product_id = p.product_id
datal=# GROUP BY sc.customer_id, sc.shopping_cart_id
datal=# HAVING SUM(sc.quantity * p.price) > 100000;
customer_id | shopping_cart_id | total_amount
-----
1000 | 400 | 169670
528 | 927 | 120780
438 | 89 | 158480
(3 筆資料)
```

15. 查詢創辦時間在 2000 年前的顧客以及其訂單總數

解釋：JOIN 顧客表與訂單表，篩選出創建帳戶日期在 2000 年 1 月 1 日前的顧客，並 COUNT 其訂單數量，按訂單總數降序排列。

```
data=# -- 14. 查詢創辦時間在 2000 年前的顧客以及其訂單總數 :
data=# SELECT c.customer_id, COUNT(o.order_id) AS order_count
data=# FROM "customer" c
data=# JOIN "order" o ON c.customer_id = o.customer_id
data=# WHERE c.account_creation_date < '2021-01-01'
data=# GROUP BY c.customer_id
data=# ORDER BY order_count DESC;
customer_id | order_count
-----
3000 | 19
589 | 2
461 | 2
239 | 2
401 | 1
476 | 1
229 | 1
481 | 1
119 | 1
258 | 1
162 | 1
757 | 1
364 | 1
220 | 1
169 | 1
730 | 1
548 | 1
217 | 1
509 | 1
506 | 1
415 | 1
474 | 1
508 | 1
268 | 1
722 | 1
196 | 1
600 | 1
128 | 1
790 | 1
427 | 1
568 | 1
(31 筆資料)
```

陸、資料載入

一、資料載入腳本

下圖為每個資料表的資料總數

table_name	row_count
inventory	791
customer	1976
shopping_cart	580
product_category	350
order	152
product	791
supplier	201
employee	943
transaction	136
order_detail	152
review	2952

(11 筆資料)

下圖為顧客資料表的匯入過程

```
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (466, 86, '2020-02-06 13:38:35', 1, '不錯');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (494, 205, '2021-10-22 04:41:13', 2, '不錯');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (790, 606, '2020-11-14 18:16:28', 1, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (124, 624, '2021-07-23 22:12:23', 3, '可以');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (281, 188, '2021-03-23 17:11:40', 4, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (441, 19, '2020-05-08 16:22:54', 1, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (949, 657, '2021-01-05 08:40:06', 1, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (907, 76, '2020-09-08 04:37:43', 5, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (604, 585, '2020-07-22 15:26:13', 2, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (211, 12, '2021-06-09 09:37:29', 2, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (183, 173, '2021-04-11 06:22:16', 1, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (115, 11, '2021-04-11 06:22:16', 1, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (445, 219, '2021-05-13 06:30:51', 2, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (115, 83, '2021-07-17 11:06:18', 3, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (415, 672, '2020-12-03 02:14:04', 3, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (402, 29, '2021-02-23 06:18:13', 4, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (282, 642, '2021-05-10 16:18:10', 5, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (1279, 768, '2021-12-21 06:29:00', 3, '可以');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (403, 169, '2020-09-07 22:16:31, 4, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (443, 76, '2020-09-18 21:16:15', 2, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (419, 42, '2020-12-14 06:07:19', 4, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (440, 203, '2020-06-24 13:06:13', 3, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (772, 21, '2020-06-10 10:18:09', 1, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (449, 312, '2021-02-21 21:36:42', 3, '可以');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (507, 111, '2020-09-27 13:00:19', 3, '很好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (778, 487, '2020-08-15 15:37:21', 2, '不好');
INSERT INTO public."review" ("customer_id", "product_id", "date", "rating", "content") VALUES (778, 487, '2020-08-15 15:37:21', 2, '不好');
```

二、範例資料展示

1. 員工 (employee)

模擬一個線上購物平台的員工資料，員工編號 2 號曾經擔任銷售專員，現在已離職 (status 0 為離職)。

data=# SELECT					
employee_id	name	phone	email	position	status
1	王世育	0955342356	pavel364@gmail.com	網路店舖經理	1
2	馬鳳凰	0937401625	evans6238@gmail.com	銷售專員	0
3	蔡德理	0922197966	franco752@gmail.com	客服專員	1
4	柯忠恩	0960586172	trevor3246@outlook.com	商品管理人	1
5	黃芝詠	0914979991	marilyn642@gmail.com	物流協調員	1
6	程坤城	0928353596	mceail781@outlook.com	資訊分析師	1
7	張凡亞	0938833891	natalie2667@yahoo.com	營運經理	0
8	許千	0930287968	jack131958@outlook.com	財務主管	1
9	程冠誠	0963486026	brian3514@yahoo.com	內容創作者	1
11	謝奇維	0925804218	nolan6176@yahoo.com	網路推廣經理	1
12	翁希森	0925165411	rogers3346@yahoo.com	產品經理	0
13	董紫舒	0926896591	deborah4375@gmail.com	品牌專員	1
15	鄭曉曉	0938411189	antonio7283@gmail.com	網站設計師	1
16	王淑維	0970697297	vanessa9512@yahoo.com	使用者體驗設計師	1
17	李妹瑞	0927768040	barlow7940@hotmail.com	系統管理員	0
18	許愛愛	0958332813	gina7146@gmail.com	社群媒體管理員	1
19	楊潤輝	0918433954	gregory8297@gmail.com	採購專員	0
20	何東遙	0952982925	brody3251@outlook.com	商務發展經理	1
21	王俊彤	0927711237	chloe5344@yahoo.com	客戶關係經理	1
22	楊慈玉	0958823131	clements567@icloud.com	內容編輯	0

2. 訂單 (order)

模擬顧客下單過程，例如顧客編號 364 號已完成一筆訂單，且該訂單折扣了 234 元。

data=# SELECT * FROM public.order LIMIT 20;							
order_id	customer_id	employee_id	shopping_cart_id	date	status	address	discount
1	364	63	77	2020-04-12 12:33:45	已完成	彰化縣溪州鄉中港路62號4樓	234
148	162	11	65	2023-04-03 09:16:15	待處理	高雄市鳳山區望山街83號之11	0
149	759	92	680	2022-06-11 08:33:18	已完成	新竹縣竹東鎮大林路83號之20	0
7	362	93	36	2023-09-24 10:28:21	待處理	新北市三區區集賢路44號5樓	0
8	989	25	663	2023-01-24 02:38:17	待處理	高雄市梧棲區石碇中興六巷23號之16	0
11	757	65	703	2023-06-28 22:09:35	處理中	高雄市三區區中街42號217	0
152	171	94	277	2021-08-04 21:38:29	處理中	臺中市北區區寶島北二巷37號之11	0
13	291	48	157	2023-02-15 13:41:42	處理中	花蓮縣鳳林鎮林森路99號4樓	0
14	195	57	458	2022-01-18 09:50:20	待處理	高雄市前鎮區知傑街59號7樓	0
16	85	96	374	2021-01-21 01:31:51	待處理	金門縣金沙鎮大武社98號8樓	0
155	42	5	547	2024-01-18 17:58:34	待處理	臺中市東區區達仁街22號	0
18	497	13	658	2022-07-28 18:35:40	待處理	新北市瑞芳區大港路有雅新村27號	0
21	481	34	245	2022-09-17 02:28:16	處理中	新竹縣尖石鄉柏福9號之20	0
23	263	4	313	2023-07-23 16:57:40	處理中	屏東縣屏東市廣福街93號	0
159	217	15	57	2024-08-18 05:55:02	處理中	臺中市北區區大港二街35號10樓	0
25	119	58	42	2021-06-03 00:06:52	處理中	高雄市林園區中區三路1號	0
26	239	10	650	2022-10-18 06:42:22	處理中	花蓮縣吉安鄉區崇三街49號	0
27	509	35	152	2024-09-02 13:45:33	待處理	臺南市王爺區區林森97號	1793
31	24	84	695	2022-01-27 00:49:25	處理中	雲林縣西螺鎮區區79號	144
32	415	41	528	2021-09-23 07:30:10	待處理	彰化縣彰化市廣善路78號	0

3. 評論(review)

模擬真實顧客在購買商品後的評價行為，可用於分析商品受歡迎程度及改進方向。

data=#	SELECT *	FROM public.review	LIMIT 20;		
customer_id	product_id	date	rating	content	
851	70	2022-09-15 23:38:12	2	不划算	
188	337	2021-02-17 18:07:24	5	划算	
63	330	2023-05-18 12:42:37	3	還行	
385	613	2021-08-08 03:38:37	4	划算	
716	145	2024-11-22 11:57:55	5	好用	
655	444	2023-08-30 01:42:12	5	好用	
656	86	2024-02-08 13:28:35	1	不划算	
994	280	2021-10-22 04:41:33	2	不划算	
794	456	2024-11-14 10:16:28	1	不好用	
104	624	2021-07-23 22:12:23	3	可以	
281	108	2022-03-23 17:13:48	4	好用	
441	19	2024-05-08 16:22:56	1	不好用	
949	657	2021-01-05 00:46:06	1	很貴	
957	76	2020-09-08 04:37:43	5	很便宜	
654	585	2020-07-22 19:35:13	1	不好用	
211	12	2023-06-05 09:37:29	2	不划算	
383	173	2021-04-12 09:29:01	2	不好用	
335	11	2023-04-11 06:22:16	1	很貴	
601	219	2023-05-13 04:24:51	2	不划算	
185	83	2021-09-17 11:06:38	3	還行	

三、統計報告

1. 商品

```
data=# -- 商品統計
data=# SELECT COUNT(*) AS total_records FROM "product";
total_records
-----
(1 筆資料) 791

data=#
data=# -- 主要欄位 (如 product_id, price, description) 值範圍
data=# SELECT
data=# MIN("product_id") AS min_product_id,
data=# MAX("product_id") AS max_product_id,
data=# MIN("price") AS min_price,
data=# MAX("price") AS max_price,
data=# FROM "product";
min_product_id | max_product_id | min_price | max_price
-----
(1 筆資料) 1 | 880 | 107 | 2495

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("description") FILTER (WHERE "description" IS NULL) / COUNT(*)), 2) AS null_description
data=# FROM "product";
null_description
-----
(1 筆資料) 0.00
```

2. 評論

```
data=# -- 評論統計
data=# SELECT COUNT(*) AS total_records FROM "review";
total_records
-----
(1 筆資料) 2952

data=#
data=# -- 主要欄位 (如 customer_id, product_id, rating, content) 值範圍
data=# SELECT
data=# MIN("rating") AS min_rating,
data=# MAX("rating") AS max_rating,
data=# FROM "review";
min_rating | max_rating
-----
(1 筆資料) 1 | 5

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("content") FILTER (WHERE "content" IS NULL) / COUNT(*)), 2) AS null_content
data=# FROM "review";
null_content
-----
(1 筆資料) 0.00
```

3. 顧客

```
data=# -- 顧客統計
data=# SELECT COUNT(*) AS total_records FROM "customer";
total_records
-----
(1 筆資料) 992

data=#
data=# -- 主要欄位 (如 identification_number, name, phone, email) 值範圍
data=# SELECT
data=# MIN("customer_id") AS min_customer_id,
data=# MAX("customer_id") AS max_customer_id,
data=# MIN("account_creation_date") AS earliest_account_creation,
data=# MAX("account_creation_date") AS latest_account_creation,
data=# FROM "customer";
min_customer_id | max_customer_id | earliest_account_creation | latest_account_creation
-----
(1 筆資料) 1 | 1000 | 2020-01-03 03:41:32 | 2020-11-30 17:00:21

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("identification_number") FILTER (WHERE "identification_number" IS NULL) / COUNT(*)), 2) AS null_identification_number,
data=# ROUND(100.0 * (COUNT("name") FILTER (WHERE "name" IS NULL) / COUNT(*)), 2) AS null_name,
data=# ROUND(100.0 * (COUNT("phone") FILTER (WHERE "phone" IS NULL) / COUNT(*)), 2) AS null_phone,
data=# ROUND(100.0 * (COUNT("email") FILTER (WHERE "email" IS NULL) / COUNT(*)), 2) AS null_email,
data=# FROM "customer";
null_identification_number | null_name | null_phone | null_email
-----
(1 筆資料) 0.00 | 0.00 | 0.00 | 0.00
```

4. 供應商

```
data=# -- 供應商統計
data=# SELECT COUNT(*) AS total_records FROM "supplier";
total_records
-----
(1 筆資料) 201

data=#
data=# -- 主要欄位 (如 company_name, phone, email) 值範圍
data=# SELECT
data=# MIN("supplier_id") AS min_supplier_id,
data=# MAX("supplier_id") AS max_supplier_id,
data=# FROM "supplier";
min_supplier_id | max_supplier_id
-----
(1 筆資料) 2 | 201

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("company_name") FILTER (WHERE "company_name" IS NULL) / COUNT(*)), 2) AS null_company_name,
data=# ROUND(100.0 * (COUNT("phone") FILTER (WHERE "phone" IS NULL) / COUNT(*)), 2) AS null_phone,
data=# ROUND(100.0 * (COUNT("email") FILTER (WHERE "email" IS NULL) / COUNT(*)), 2) AS null_email,
data=# FROM "supplier";
null_company_name | null_phone | null_email
-----
(1 筆資料) 0.00 | 0.00 | 0.00
```

5. 訂單

```
data=# -- 訂單統計
data=# SELECT COUNT(*) AS total_records FROM "order";
total_records
-----
(1 筆資料) 152

data=#
data=# -- 主要欄位 (如 customer_id, employee_id, shopping_cart_id) 值範圍
data=# SELECT
data=# MIN("order_id") AS min_order_id,
data=# MAX("order_id") AS max_order_id,
data=# MIN("date") AS earliest_order_date,
data=# MAX("date") AS latest_order_date,
data=# FROM "order";
min_order_id | max_order_id | earliest_order_date | latest_order_date
-----
(1 筆資料) 1 | 495 | 2021-01-02 06:58:03 | 2024-11-29 16:09:01

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("customer_id") FILTER (WHERE "customer_id" IS NULL) / COUNT(*)), 2) AS null_customer_id,
data=# ROUND(100.0 * (COUNT("employee_id") FILTER (WHERE "employee_id" IS NULL) / COUNT(*)), 2) AS null_employee_id,
data=# ROUND(100.0 * (COUNT("shopping_cart_id") FILTER (WHERE "shopping_cart_id" IS NULL) / COUNT(*)), 2) AS null_shopping_cart_id,
data=# FROM "order";
null_customer_id | null_employee_id | null_shopping_cart_id
-----
(1 筆資料) 0.00 | 0.00 | 0.00
```

6. 訂單明細

```
data=# -- 訂單明細統計
data=# SELECT COUNT(*) AS total_records FROM "order_detail";
total_records
-----
(1 筆資料) 152

data=#
data=# -- 主要欄位 (如 customer_id, order_id, product_id) 值範圍
data=# SELECT
data=# MIN("customer_id") AS min_customer_id,
data=# MAX("customer_id") AS max_customer_id,
data=# MIN("order_id") AS min_order_id,
data=# MAX("order_id") AS max_order_id,
data=# FROM "order_detail";
min_customer_id | max_customer_id | min_order_id | max_order_id
-----
(1 筆資料) 6 | 800 | 1 | 495

data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("quantity") FILTER (WHERE "quantity" IS NULL) / COUNT(*)), 2) AS null_quantity,
data=# FROM "order_detail";
null_quantity
-----
(1 筆資料) 0.00
```

7. 商品類別

```
data=# -- 總記錄數
data=# SELECT COUNT(*) AS total_records FROM "product_category";
total_records
-----
350
(1 筆資料)
```

```
data=#
data=# -- 主要欄位 (如 name) 值範圍
data=# SELECT
data=# MIN("product_category_id") AS min_category_id,
data=# MAX("product_category_id") AS max_category_id
data=# FROM "product_category";
min_category_id | max_category_id
-----
1 | 350
(1 筆資料)
```

```
data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("name") FILTER (WHERE "name" IS NULL) / COUNT(*)), 2) AS null_name
data=# FROM "product_category";
null_name
-----
0.00
(1 筆資料)
```

8. 交易

```
data=# -- 總記錄數
data=# SELECT COUNT(*) AS total_records FROM "transaction";
total_records
-----
136
(1 筆資料)
```

```
data=#
data=# -- 主要欄位 (如 customer_id, order_id, method) 值範圍
data=# SELECT
data=# MIN("transaction_id") AS min_transaction_id,
data=# MAX("transaction_id") AS max_transaction_id
data=# FROM "transaction";
min_transaction_id | max_transaction_id
-----
3 | 138
(1 筆資料)
```

```
data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("method") FILTER (WHERE "method" IS NULL) / COUNT(*)), 2) AS null_method
data=# FROM "transaction";
null_method
-----
0.00
(1 筆資料)
```

9. 購物車

```
data=# -- 總記錄數
data=# SELECT COUNT(*) AS total_records FROM "shopping_cart";
total_records
-----
579
(1 筆資料)
```

```
data=#
data=# -- 主要欄位 (如 customer_id, product_id, quantity) 值範圍
data=# SELECT
data=# MIN("shopping_cart_id") AS min_cart_id,
data=# MAX("shopping_cart_id") AS max_cart_id,
data=# MIN("quantity") AS min_quantity,
data=# MAX("quantity") AS max_quantity
data=# FROM "shopping_cart";
min_cart_id | max_cart_id | min_quantity | max_quantity
-----
2 | 999 | 1 | 95
(1 筆資料)
```

```
data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("quantity") FILTER (WHERE "quantity" IS NULL) / COUNT(*)), 2) AS null_quantity
data=# FROM "shopping_cart";
null_quantity
-----
0.00
(1 筆資料)
```

10. 庫存

```
data=# -- 總記錄數
data=# SELECT COUNT(*) AS total_records FROM "inventory";
total_records
-----
791
(1 筆資料)
```

```
data=#
data=# -- 主要欄位 (如 product_id, quantity, location) 值範圍
data=# SELECT
data=# MIN("inventory_id") AS min_inventory_id,
data=# MAX("inventory_id") AS max_inventory_id,
data=# MIN("quantity") AS min_quantity,
data=# MAX("quantity") AS max_quantity
data=# FROM "inventory";
min_inventory_id | max_inventory_id | min_quantity | max_quantity
-----
1 | 800 | 1 | 250
(1 筆資料)
```

```
data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("quantity") FILTER (WHERE "quantity" IS NULL) / COUNT(*)), 2) AS null_quantity,
data=# ROUND(100.0 * (COUNT("location") FILTER (WHERE "location" IS NULL) / COUNT(*)), 2) AS null_location
data=# FROM "inventory";
null_quantity | null_location
-----
0.00 | 0.00
(1 筆資料)
```

11. 員工

```
data=# -- 總記錄數
data=# SELECT COUNT(*) AS total_records FROM "employee";
total_records
-----
100
(1 筆資料)
```

```
data=#
data=# -- 主要欄位 (如 name, phone, email) 值範圍
data=# SELECT
data=# MIN("employee_id") AS min_employee_id,
data=# MAX("employee_id") AS max_employee_id
data=# FROM "employee";
min_employee_id | max_employee_id
-----
1 | 100
(1 筆資料)
```

```
data=#
data=# -- 空值比例
data=# SELECT
data=# ROUND(100.0 * (COUNT("name") FILTER (WHERE "name" IS NULL) / COUNT(*)), 2) AS null_name,
data=# ROUND(100.0 * (COUNT("phone") FILTER (WHERE "phone" IS NULL) / COUNT(*)), 2) AS null_phone,
data=# ROUND(100.0 * (COUNT("email") FILTER (WHERE "email" IS NULL) / COUNT(*)), 2) AS null_email
data=# FROM "employee";
null_name | null_phone | null_email
-----
0.00 | 0.00 | 0.00
(1 筆資料)
```

柒、測試與查詢優化

一、功能性測試報告

1. CRUD

新增一筆員工編號為 101 的員工資料，並查詢電話為 0912345678 的員工資料。

```
data=# INSERT INTO "employee" ("name", "phone", "email", "position", "status") VALUES ('張三', '0912345678', 'zhang@company.com', '業務員', 1);
INSERT 0 1
data=# SELECT * FROM "employee" WHERE "phone"='0912345678';
 employee_id | name | phone | email | position | status
-----
      101 | 張三 | 0912345678 | zhang@company.com | 業務員 | 1
(1 筆資料)
```

更新電話為 0912345678 的員工資料，把職位更新為高級業務員。

```
data=# UPDATE "employee" SET "position" = '高級業務員' WHERE "phone" = '0912345678';
UPDATE 1
data=# SELECT * FROM "employee" WHERE "phone"='0912345678';
 employee_id | name | phone | email | position | status
-----
      101 | 張三 | 0912345678 | zhang@company.com | 高級業務員 | 1
(1 筆資料)
```

刪除電話為 0912345678 的員工資料。

```
data=# DELETE FROM "employee" WHERE "phone" = '0912345678';
DELETE 1
data=# SELECT * FROM "employee" WHERE "phone"='0912345678';
 employee_id | name | phone | email | position | status
-----
(0 筆資料)
```

2. 主鍵與外鍵完整限制測試：customer_id 同時是 order_detail 表的主鍵與外鍵，新增的 customer_id 不存在於 customer 表，會出現錯誤。

```
data=# INSERT INTO "order_detail" ("customer_id", "order_id", "product_id", "quantity") VALUES (1, 1, 1, 2);
INSERT 0 1
data=# SELECT * FROM "order_detail" WHERE "order_id"=1;
 customer_id | order_id | product_id | quantity
-----
          1 |         1 |           1 |         2
(1 筆資料)

data=# INSERT INTO "order_detail" ("customer_id", "order_id", "product_id", "quantity") VALUES (9999, 9999, 9999, 3);
ERROR:  insert or update on table "order_detail" violates foreign key constraint "customer_id"
詳細資料:  Key (customer_id)=(9999) is not present in table "customer".
data=#
```

3. 唯一性限制測試：phone 設為 UNIQUE，新增兩個一樣的 phone 會出現錯誤。

```
data=# INSERT INTO "supplier" ("company_name", "contact_name", "phone", "email", "address") VALUES ('範例公司', '王先生', '0212345678', 'supplier@example.com', '台北市範例路');
INSERT 0 1
data=# SELECT * FROM "supplier" WHERE "company_name"='範例公司';
 supplier_id | company_name | contact_name | phone | email | address
-----
          1 | 範例公司 | 王先生 | 0212345678 | supplier@example.com | 台北市範例路
(1 筆資料)

data=# INSERT INTO "supplier" ("company_name", "contact_name", "phone", "email", "address") VALUES ('另一家公司', '李先生', '0212345678', 'another@example.com', '新北市範例路');
ERROR:  duplicate key value violates unique constraint "supplier_phone_key"
詳細資料:  Key (phone)=(0212345678) already exists.
data=#
```

4. 檢查限制測試：評論商品只能輸入 1-5 分，超過 1-5 分則會出現錯誤。

```
data=# \d review
資料表 "public.review"
  欄位 |          型別          | 定序 | 能否為 NULL | 預設
-----+-----+-----+-----+-----
customer_id | integer                |      | not null     |
product_id  | integer                |      | not null     |
date        | timestamp without time zone |      | not null     |
rating      | integer                |      | not null     |
content     | text                   |      | not null     |
索引:
    "review_pkey" PRIMARY KEY, btree (customer_id, product_id)
檢查規則:
    "rating" CHECK (rating >= 1 AND rating <= 5)
外發語言規則:
    "customer_id" FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
    "product_id" FOREIGN KEY (product_id) REFERENCES product(product_id)

data=# INSERT INTO "public"."review" ("customer_id", "product_id", "date", "rating", "content") VALUES ('5', '5', '2024-12-17 03:09:56', -1, '超級差評');
ERROR:  new row for relation "review" violates check constraint "rating"
詳細資料:  Failing row contains (5, 5, 2024-12-17 03:09:56, -1, 超級差評).
```

二、查詢效能報告(EXPLAIN 或 EXPLAIN ANALYZE)

EXPLAIN ANALYZE 會生成查詢的執行計劃和實際執行查詢，並記錄查詢的各階段執行所需的時間，這使得開發人員能夠將預估成本與真實執行時間進行比較，進而診斷查詢效能瓶頸。下方圖片為 15 個複雜查詢中第 2、3 和 5 題的 EXPLAIN ANALYZE 結果。

● 查詢信用卡交易成功次數最多的前五名顧客

```
data=# EXPLAIN ANALYZE
data=# SELECT public.customer.customer_id, COUNT(transaction.transaction_id) AS total_success_transactions
data=# FROM public.customer
data=# JOIN public.transaction ON public.customer.customer_id = public.transaction.customer_id
data=# WHERE public.transaction.method = '信用卡' AND public.transaction.status = '成功'
data=# GROUP BY public.customer.customer_id
data=# ORDER BY total_success_transactions DESC
data=# LIMIT 5;

QUERY PLAN

Limit (cost=44.40..44.41 rows=5 width=12) (actual time=0.140..0.141 rows=5 loops=1)
  -> Sort (cost=44.40..44.41 rows=18 width=12) (actual time=0.140..0.141 rows=5 loops=1)
        Sort Key: (count(transaction.transaction_id)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        -> GroupAggregate (cost=43.78..44.10 rows=18 width=12) (actual time=0.132..0.135 rows=17 loops=1)
              Group Key: customer.customer_id
              -> Sort (cost=43.78..43.83 rows=18 width=8) (actual time=0.129..0.131 rows=20 loops=1)
                    Sort Key: customer.customer_id
                    Sort Method: quicksort  Memory: 25kB
                    -> Hash Join (cost=39.32..43.41 rows=18 width=8) (actual time=0.113..0.124 rows=20 loops=1)
                          Hash Cond: (transaction.customer_id = customer.customer_id)
                          -> Seq Scan on transaction (cost=0.00..4.04 rows=18 width=8) (actual time=0.007..0.016 rows=20 loops=1)
                                Filter: ((method = '信用卡'::text) AND (status = '成功'::text))
                                Rows Removed by Filter: 116
                          -> Seq Scan on customer (cost=0.00..26.92 rows=992 width=4) (actual time=0.100..0.100 rows=992 loops=1)
                                Buckets: 1024 Batches: 1 Memory Usage: 43kB
                                -> Seq Scan on customer (cost=0.00..26.92 rows=992 width=4) (actual time=0.003..0.058 rows=992 loops=1)

Planning Time: 0.158 ms
Execution Time: 0.161 ms
(19 筆資料)
```

● 查詢在職員工處理的訂單數量排名

```
data=# EXPLAIN ANALYZE
data=# SELECT public.order.employee_id, COUNT(public.order.order_id) AS order_count
data=# FROM public.order
data=# JOIN public.employee ON public.order.employee_id = public.employee.employee_id
data=# WHERE public.employee.status = 1
data=# GROUP BY public.order.employee_id
data=# ORDER BY order_count DESC;

QUERY PLAN

Sort (cost=10.26..10.26 rows=2 width=12) (actual time=0.090..0.091 rows=67 loops=1)
  Sort Key: (count("order".order_id)) DESC
  Sort Method: quicksort  Memory: 27kB
  -> GroupAggregate (cost=10.21..10.25 rows=2 width=12) (actual time=0.067..0.083 rows=67 loops=1)
        Group Key: "order".employee_id
        -> Sort (cost=10.21..10.22 rows=2 width=8) (actual time=0.065..0.069 rows=152 loops=1)
              Sort Key: "order".employee_id
              Sort Method: quicksort  Memory: 29kB
              -> Hash Join (cost=3.26..10.20 rows=2 width=8) (actual time=0.029..0.051 rows=152 loops=1)
                    Hash Cond: ("order".employee_id = employee.employee_id)
                    -> Seq Scan on "order" (cost=0.00..6.52 rows=152 width=8) (actual time=0.005..0.012 rows=152 loops=1)
                    -> Hash (cost=3.25..3.25 rows=1 width=4) (actual time=0.015..0.015 rows=77 loops=1)
                          Buckets: 1024 Batches: 1 Memory Usage: 11kB
                          -> Seq Scan on employee (cost=0.00..3.25 rows=1 width=4) (actual time=0.005..0.011 rows=77 loops=1)
                                Filter: (status = 1)
                                Rows Removed by Filter: 23

Planning Time: 0.122 ms
Execution Time: 0.107 ms
(18 筆資料)
```

● 查詢評分至少 4 分且被評價至少 3 次的商品

```
data=# EXPLAIN ANALYZE
data=# SELECT product_id, rating_count
data=# FROM (
data=#   SELECT product_id, COUNT(rating) AS rating_count
data=#   FROM public.review
data=#   WHERE rating >= 4
data=#   GROUP BY product_id
data=#   HAVING COUNT(rating) >= 3
data=# ) AS subquery
data=# ORDER BY product_id;

QUERY PLAN

Sort (cost=81.78..82.33 rows=221 width=12) (actual time=0.282..0.285 rows=145 loops=1)
  Sort Key: review.product_id
  Sort Method: quicksort  Memory: 30kB
  -> HashAggregate (cost=64.89..73.17 rows=221 width=12) (actual time=0.240..0.269 rows=145 loops=1)
        Group Key: review.product_id
        Filter: (count(review.rating) >= 3)
        Batches: 1 Memory Usage: 105kB
        Rows Removed by Filter: 483
        -> Seq Scan on review (cost=0.00..58.90 rows=1197 width=8) (actual time=0.008..0.133 rows=1197 loops=1)
              Filter: (rating >= 4)
              Rows Removed by Filter: 1755

Planning Time: 0.061 ms
Execution Time: 0.310 ms
(13 筆資料)
```

三、壓力測試結果

下方圖片為壓力測試圖，展示了系統在壓力測試下的響應時間和效能表現。響應時間數據（如最小、平均、最大和 95 百分位延遲）有助於了解系統在不同負載條件下的性能，效能數據（如查詢數量、事務數量和錯誤率）則有助於評估系統的吞吐量和穩定性。

```
yuan@YUAIUY-PC:~$ sysbench oltp_read_write --db-driver=pgsql \  
  --pgsql-host=127.0.0.1 --pgsql-port=5432 \  
  --pgsql-user=sbtest_user --pgsql-password=password \  
  --pgsql-db=sbtest --tables=10 --table-size=1000000 \  
  --threads=16 --time=60 run  
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)  
  
Running the test with following options:  
Number of threads: 16  
Initializing random number generator from current time  
  
Initializing worker threads...  
  
Threads started!  
  
SQL statistics:  
  queries performed:  
    read: 3844512  
    write: 1098376  
    other: 549242  
    total: 5492130  
  transactions: 274593 (4576.29 per sec.)  
  queries: 5492130 (91530.22 per sec.)  
  ignored errors: 15 (0.25 per sec.)  
  reconnects: 0 (0.00 per sec.)  
  
General statistics:  
  total time: 60.0025s  
  total number of events: 274593  
  
Latency (ms):  
  min: 1.40  
  avg: 3.49  
  max: 66.53  
  95th percentile: 6.43  
  sum: 959623.06  
  
Threads fairness:  
  events (avg/stddev): 17162.0625/18.64  
  execution time (avg/stddev): 59.9764/0.00
```